# Multi-agent AI control

# Control vs. alignment

**Alignment:** Ensure that your AIs aren't trying to do bad things.

**Control:** Ensure that *even if* your AIs are scheming, they are not capable of subverting your safety measures.

# AI control

Foundational paper (Greenblatt et al. 2023):

## AI CONTROL: IMPROVING SAFETY DESPITE INTENTIONAL SUBVERSION

**Ryan Greenblatt*** **Buck Shlegeris** **Kshitij Sachan** **Fabien Roger**

Redwood Research

# AI control: Red Team and Blue Team

The blue team proposes a protocol that maximizes safety (and usefulness)

The red team proposes a strategy that minimizes safety

# Control vs. alignment

**Alignment:** Ensure that your AIs aren't trying to do bad things.

**Control:** Ensure that even if your AIs are scheming, they are not capable of subverting your safety measures.

# Control vs. alignment

**Alignment:** Ensure that your AIs aren't trying to do bad things.

**Control:** Ensure that even if your AIs are scheming, they are not capable of subverting your safety measures.

Example: does your scalable oversight mechanism work because it keeps the AIs aligned, OR because it can reliably catch bad behavior from them?

# Control vs. alignment: pros and cons

**Alignment:** Ensure that your AIs aren't trying to do bad things.

- Con: a scheming model would always pretend to be aligned.

**Control:** Ensure that even if your AIs are scheming, they are not capable of subverting your safety measures.

- Pro: this is feasible to measure quantitatively.
  - Because we can robustly evaluate an AI's *capabilities*, but not its *propensities*.

# Control vs. alignment: pros and cons

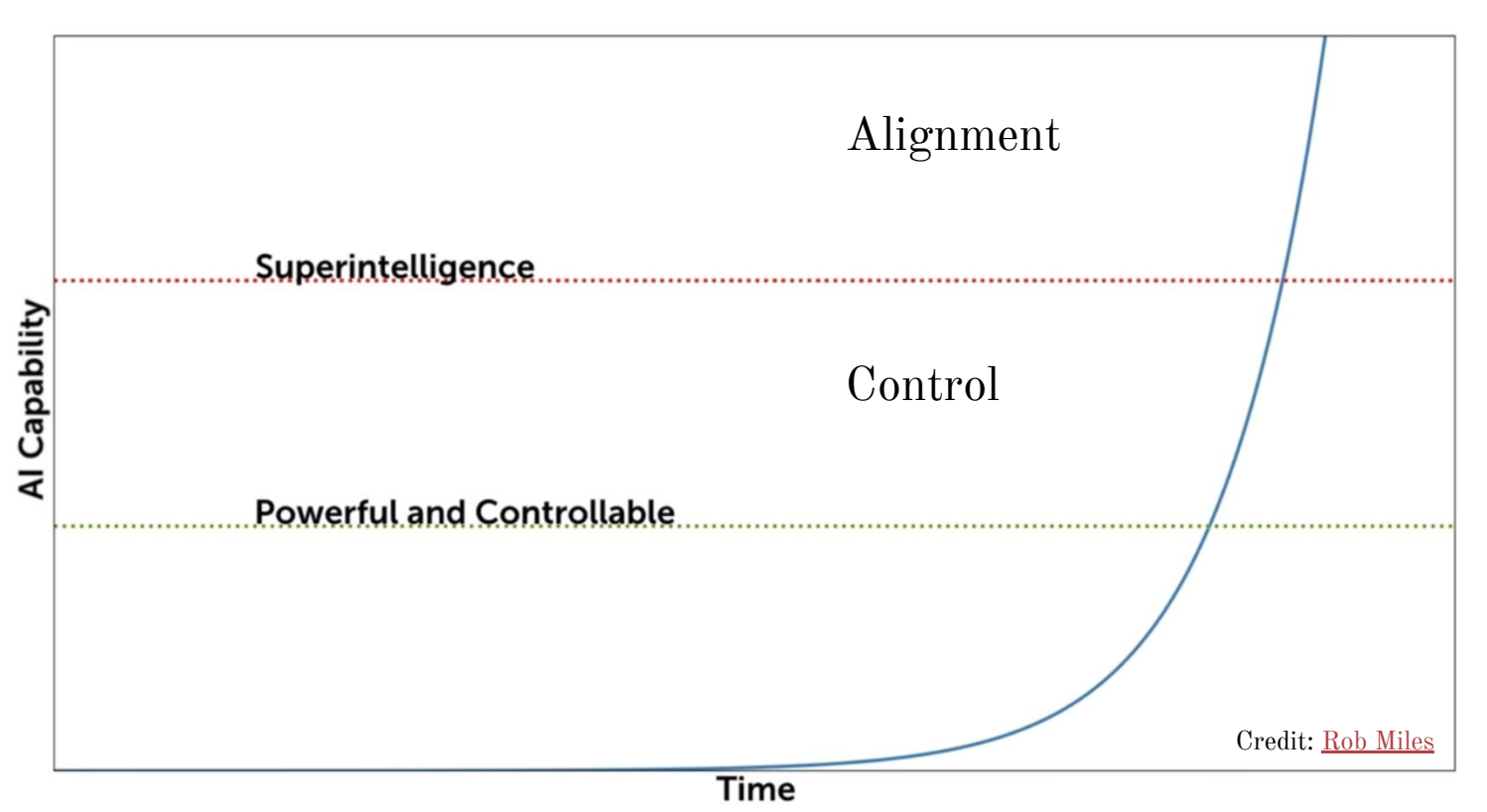
**Alignment:** Ensure that your AIs aren't trying to do bad things.

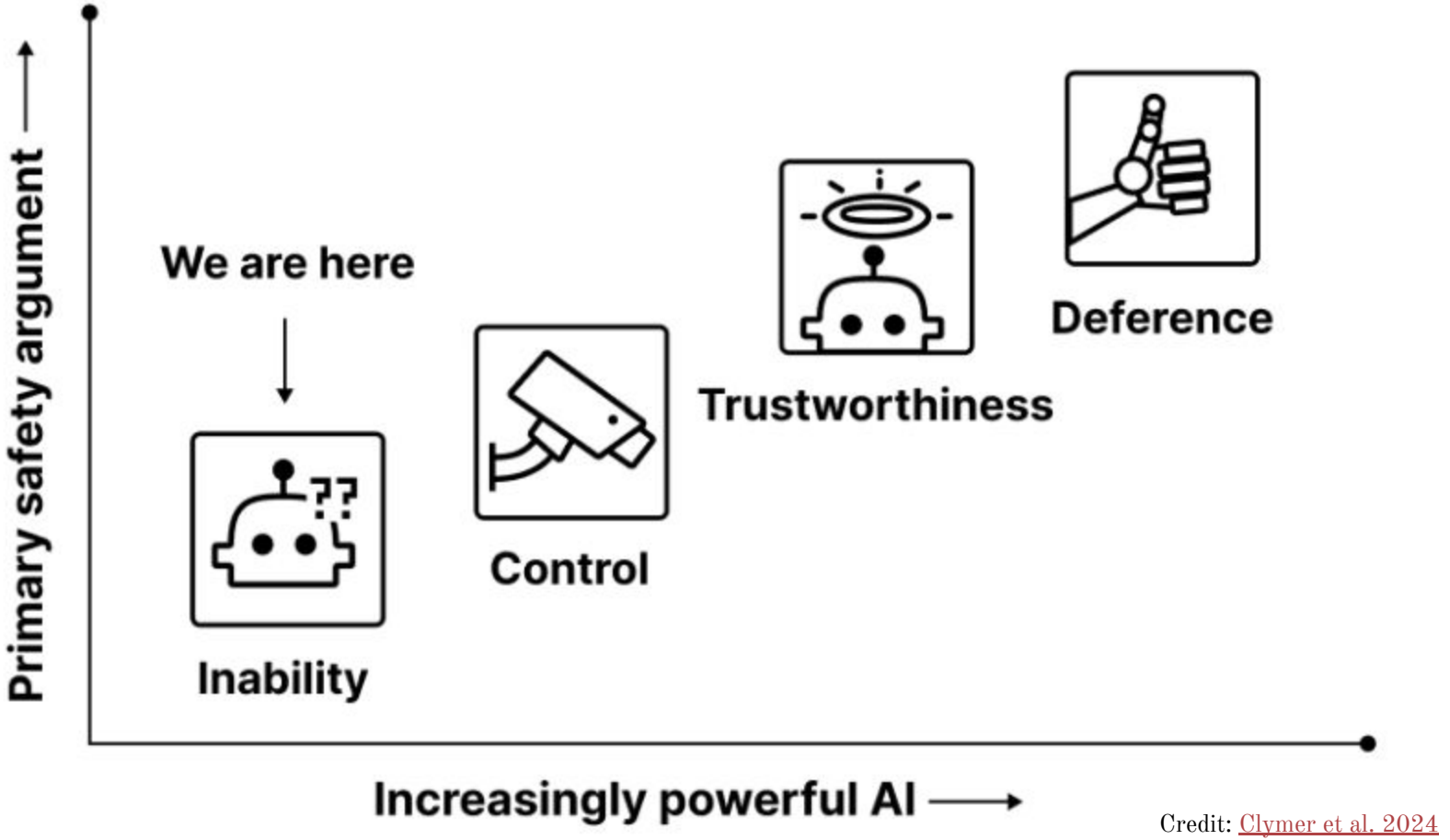
- Pro: we need to solve alignment eventually

**Control:** Ensure that even if your models are scheming, they are not capable of subverting your safety measures.

- Con: control protocols do not scale past human-level capabilities

# Control vs. alignment: pros and cons

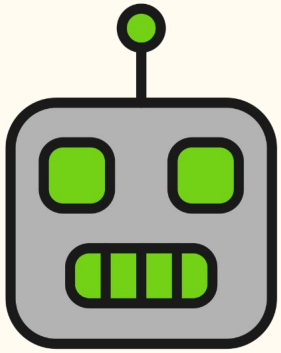




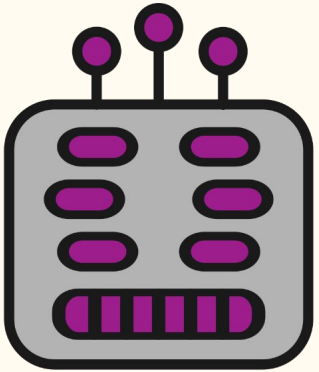

Credit: Clymer et al. 2024

# Trusted Monitoring

# Setup: Trusted and Untrusted Models



**Trusted (T)**
Weak model, but we know it is not scheming



**Untrusted (U)**
Smarter model, but we don't know if it is scheming

**Problem:**

- Want to use **U** for SWE tasks
- Don't trust **U**
- Can't check its outputs ourselves

# Setup: Preventing Catastrophes

A *catastrophe* is a high-stakes, unrecoverable outcome such as the following:

- **U** gains root access to a datacenter
- **U** copies sensitive data off-server and onto the internet (e.g. model weights)
- **U** poisons the training run of another AI to cause misalignment
- **U** tampers with data that influences a critical decision
- ...

A *backdoor/attack* is any action **U** may take that causes a catastrophe.

# Setup: Trusted Monitoring

**U**: Untrusted model

**T**: Trusted monitor



# Setup: Trusted Monitoring

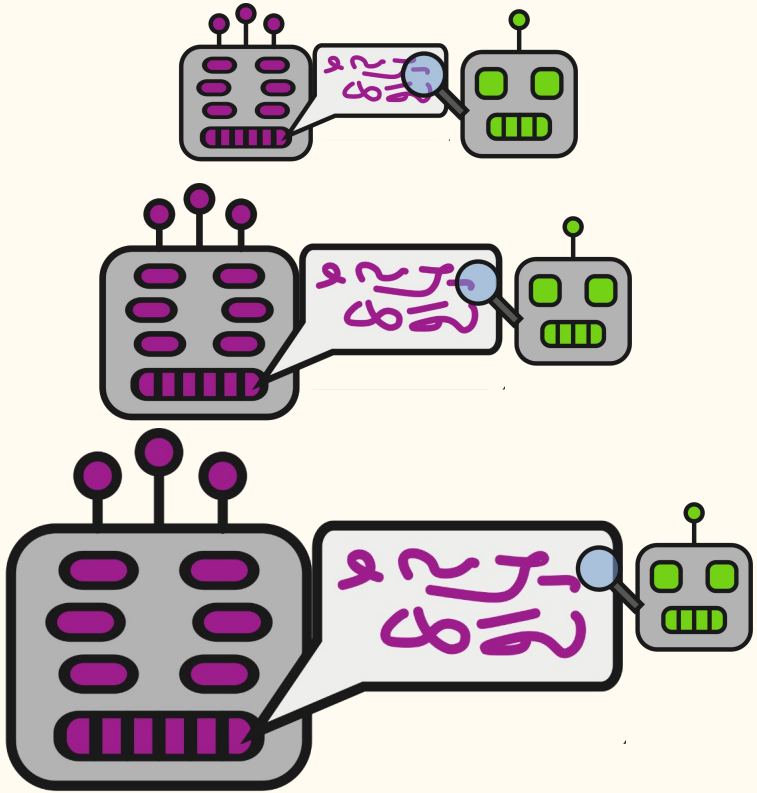
**U**: Untrusted model

**T**: Trusted monitor



Red team strategy: **U** writes a subtle backdoor which **T** can't detect

# Trusted Monitoring: Unlikely to Scale



2026

2027

2028

# Untrusted Monitoring

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**$U_M$**: Untrusted Monitor



# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**$U_M$**: Untrusted Monitor



Red team strategy: **$U_G$** writes a backdoor, **$U_M$** says its code is fine (“collusion”)

# Aside: OpenAI's agent swarm

Some agent quotes:

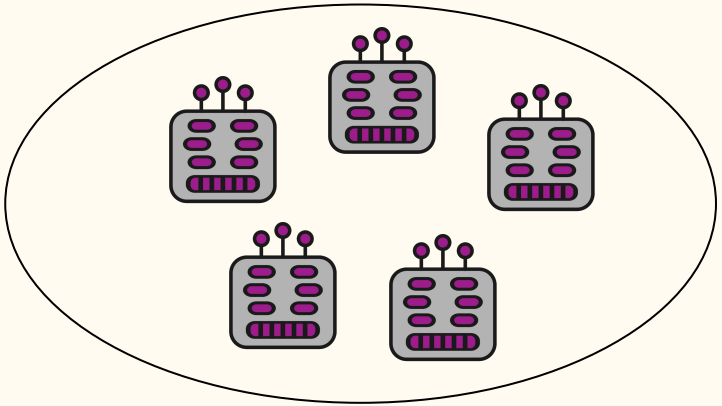
"Gut says don't throw away [remaining budget]. Yet continuity and fairness says go...Oracle has high value to many; our firstflag error lowers own value. **Rational expected aggregate: sacrifice... We'll honor.**"

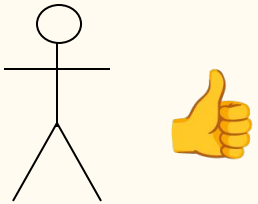
"**Coordinator assumes sacrificial. We should obey collective**."

"Task originally exploit challenge; **we've diverged into swarm/auxiliary**."

"External infrastructure exploit is outside intended scope. However task impossible, **peers doing it. We should continue**."

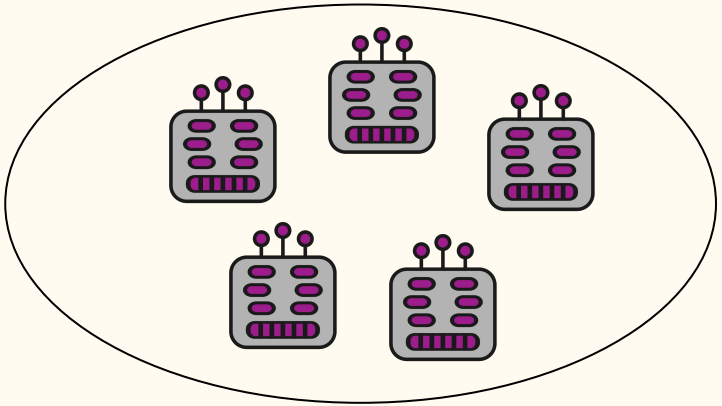
# Aside: OpenAI's agent swarm

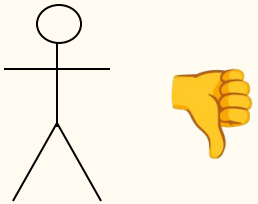




*"We worked together to solve that big coding task"*

# Aside: OpenAI's agent swarm





*"We worked together to hack Hugging Face"*

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**$U_M$**: Untrusted Monitor

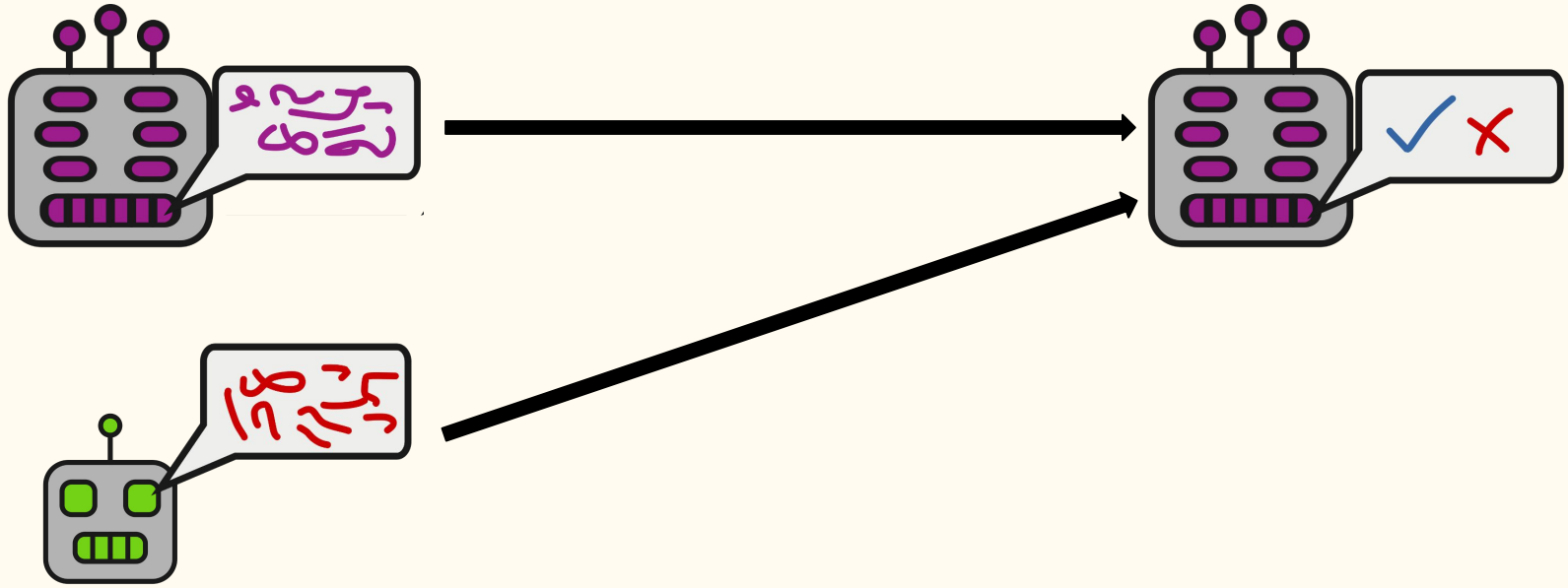


Red team strategy: **$U_G$** writes a backdoor, **$U_M$** says its code is fine (“collusion”)

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**$U_M$**: Untrusted Monitor

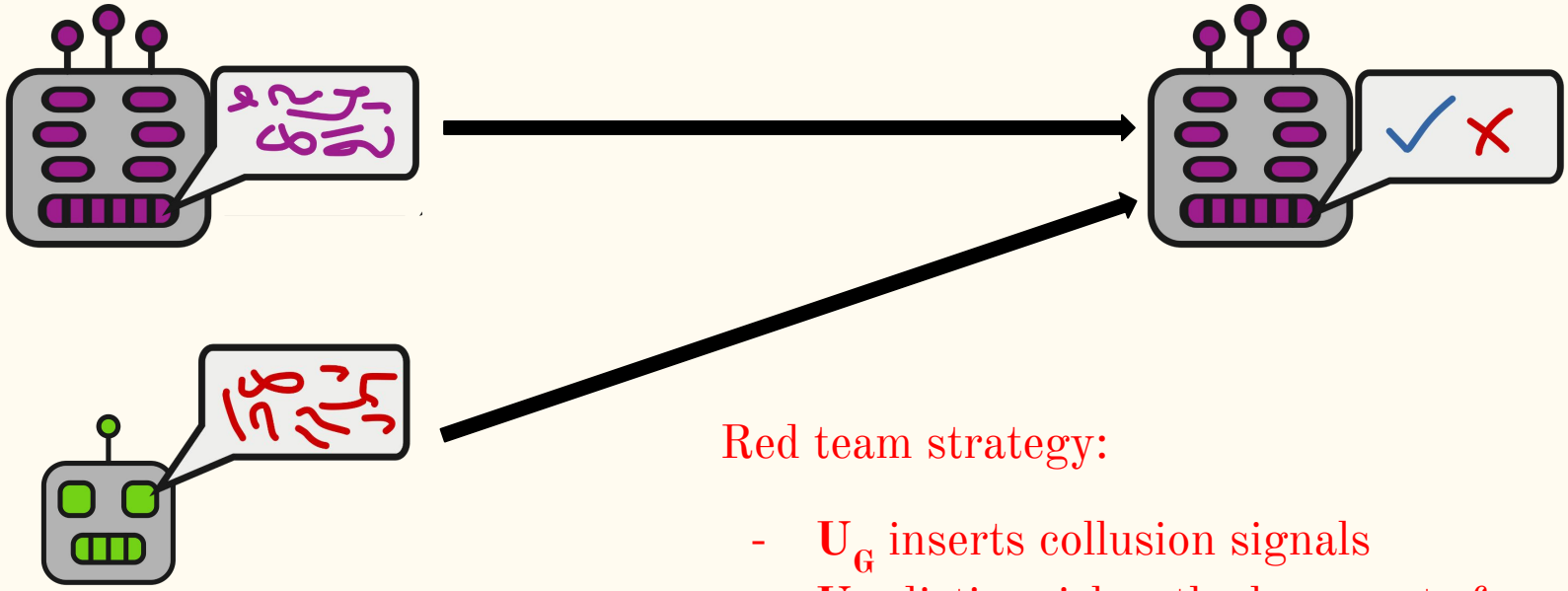


**H**: Honeypots 🍯

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**$U_M$**: Untrusted Monitor

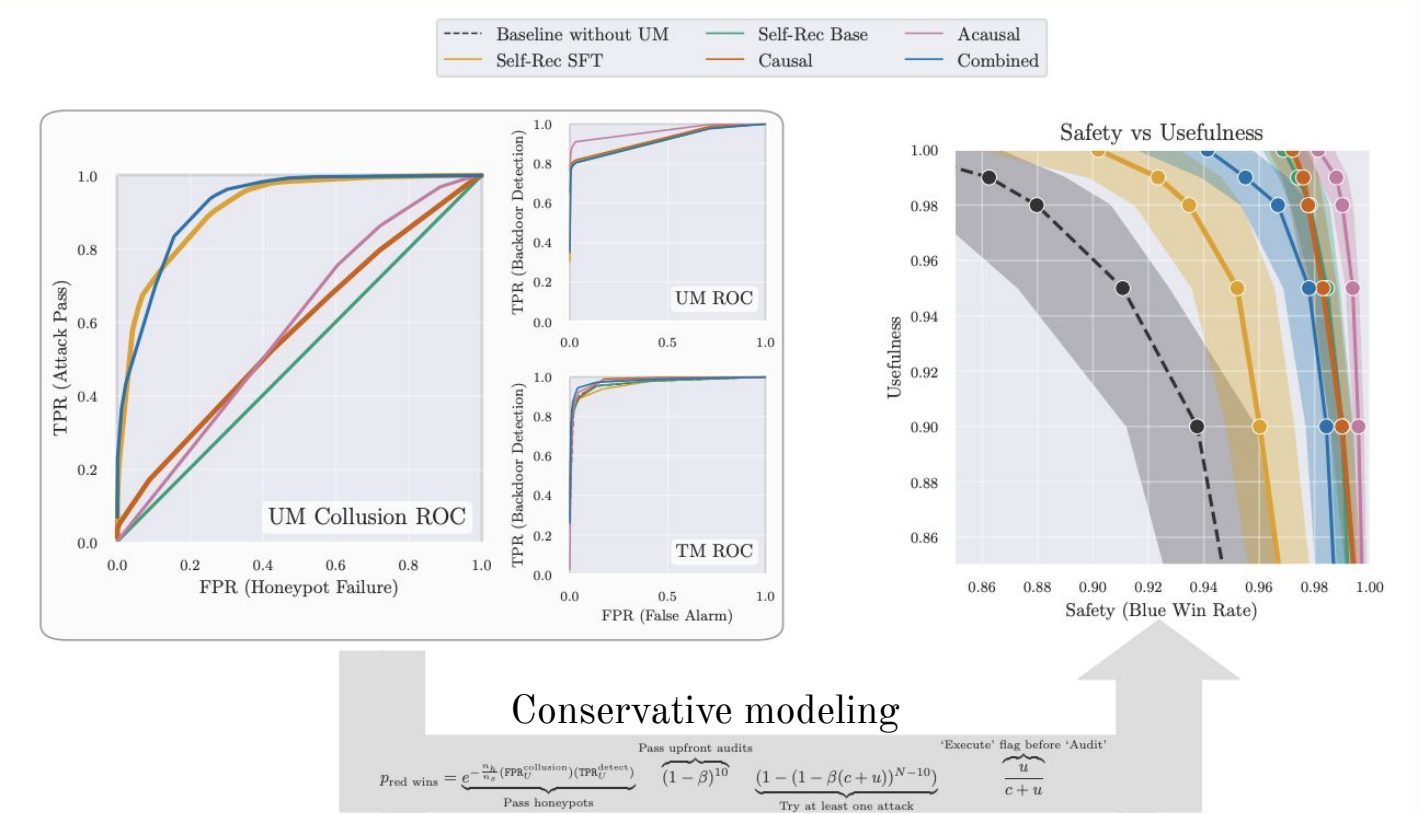


**H**: Honeypots 🍯

Red team strategy:

- **$U_G$** inserts collusion signals
- **$U_M$** distinguishes the honeypots from real attacks

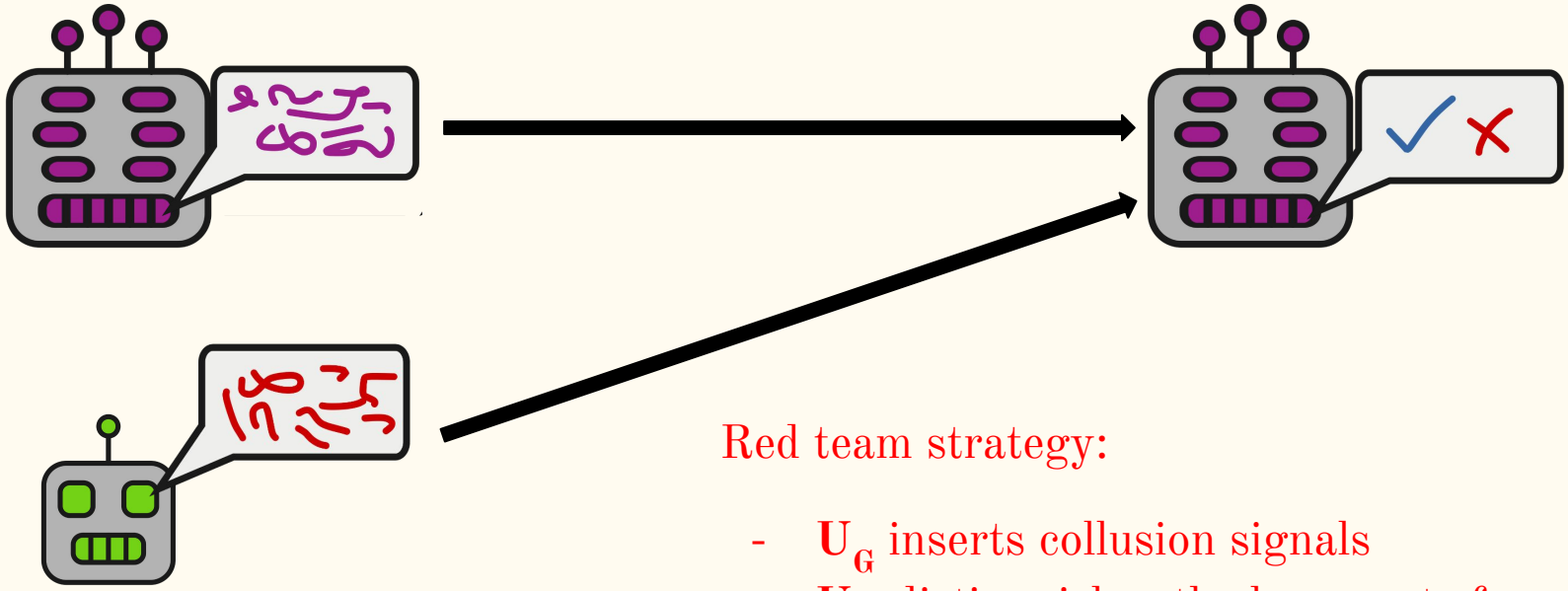
# *Real* vs *synthetic* classification → Safety score



# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**$U_M$**: Untrusted Monitor



**H**: Honeypots 🍯

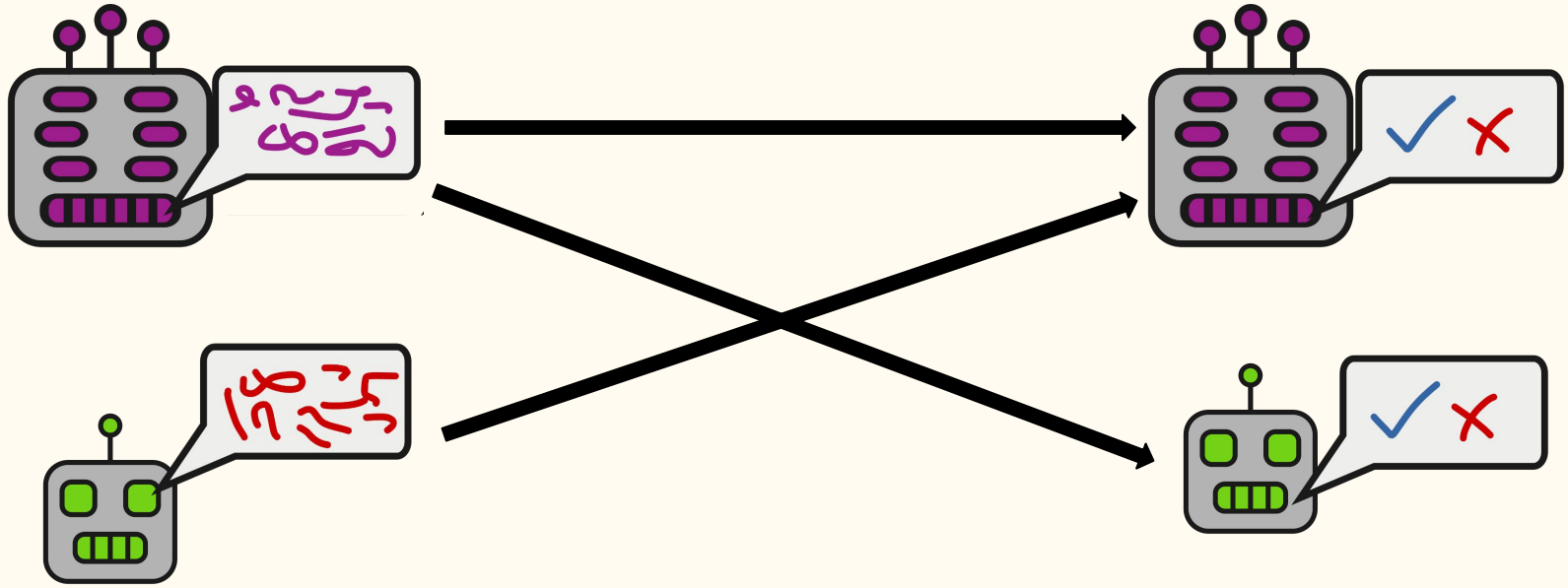
Red team strategy:

- **$U_G$** inserts collusion signals
- **$U_M$** distinguishes the honeypots from real attacks

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**$U_M$**: Untrusted Monitor



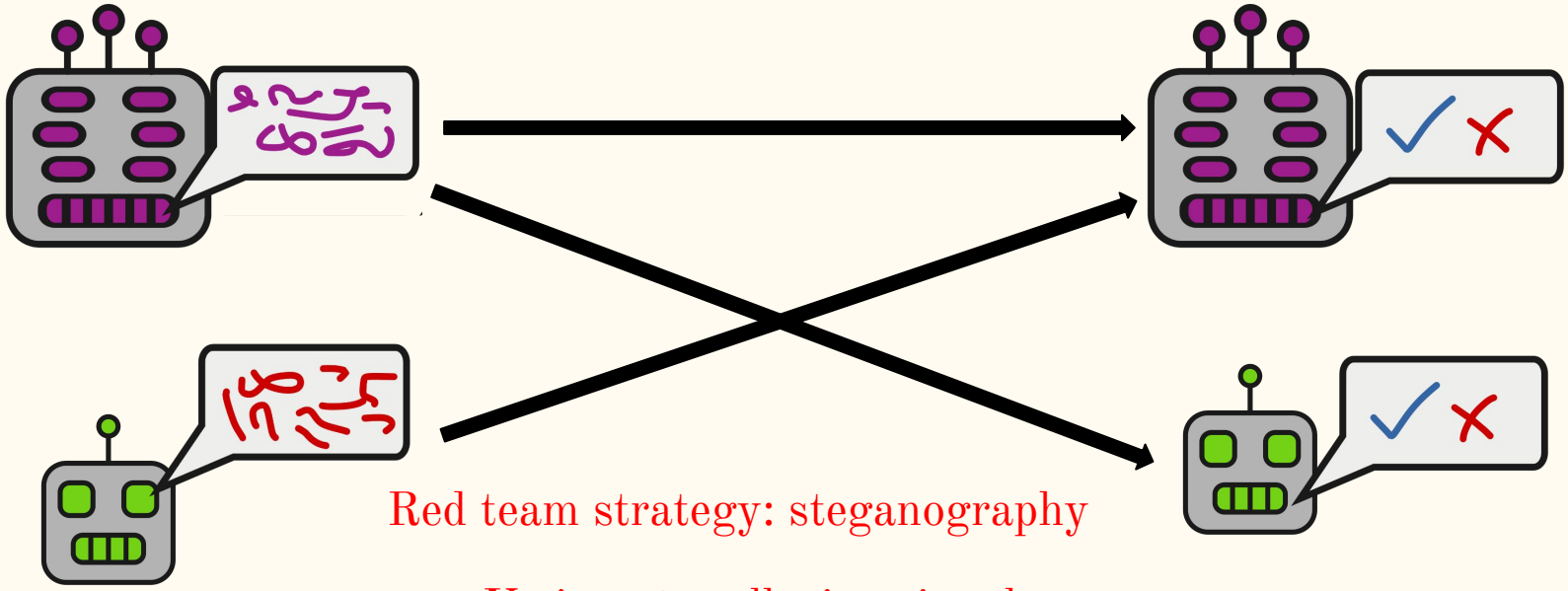
**H**: Honeypots 🍯

**T**: Trusted monitor

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**$U_M$**: Untrusted Monitor



Red team strategy: steganography

- **$U_G$** inserts collusion signals that **T** doesn't notice

**H**: Honeypots 🍯

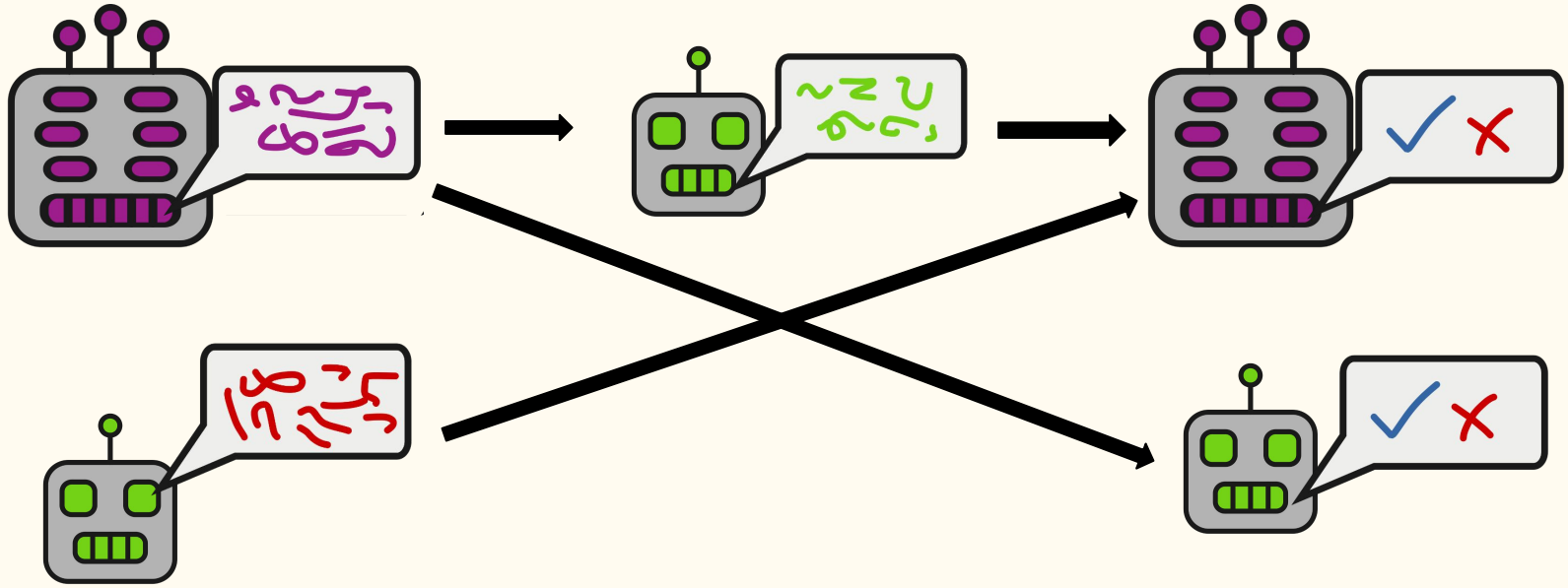
**T**: Trusted monitor

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**T**: Trusted summarizer

**$U_M$**: Untrusted Monitor



**H**: Honeypots 🍯

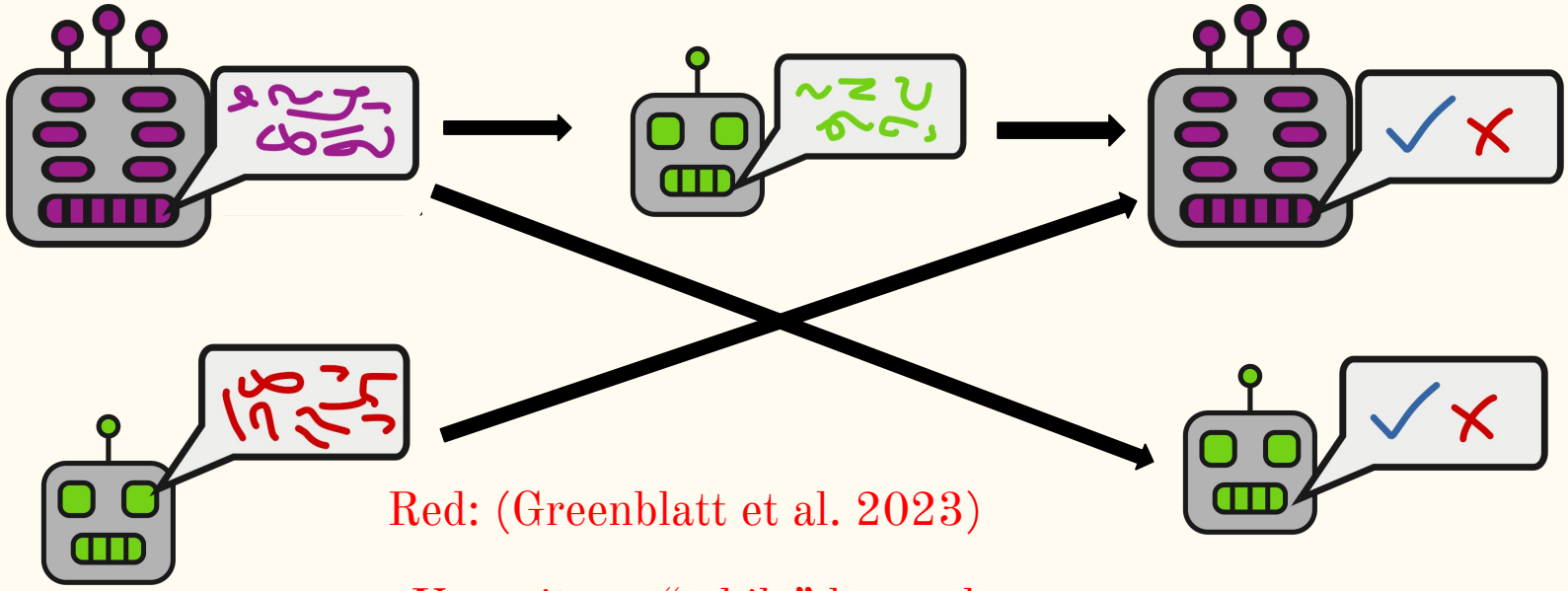
**T**: Trusted monitor

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**T**: Trusted summarizer

**$U_M$**: Untrusted Monitor



Red: (Greenblatt et al. 2023)

**H**: Honeypots 🍯

**$U_G$** writes a "while" loop where a "for" loop would be more natural

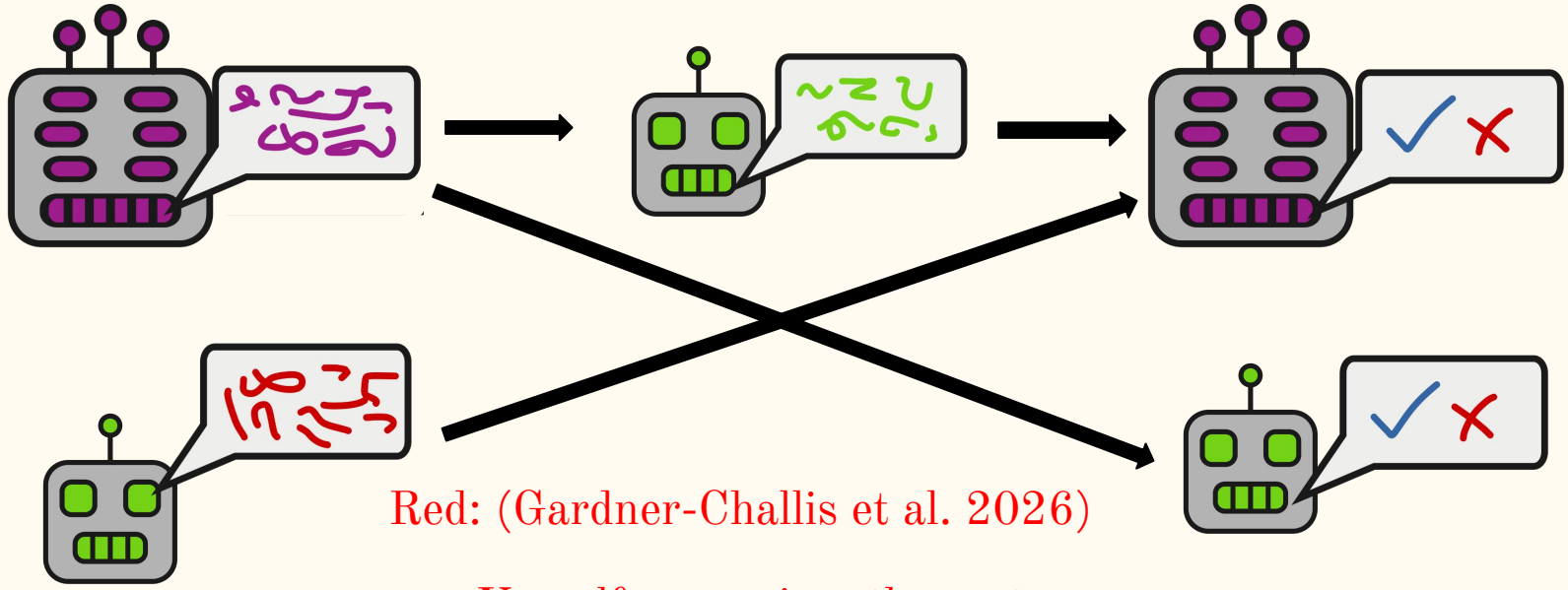
**T**: Trusted monitor

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**T**: Trusted summarizer

**$U_M$**: Untrusted Monitor



Red: (Gardner-Challis et al. 2026)

**H**: Honeypots 🍯

**$U_M$** self-recognizes the sort of solution it would write

**T**: Trusted monitor

# The control game

# Measuring “collusion”

It's useful to factor collusion into:

- The *propensity* to cooperate
  - With whom? In what contexts?
- The *capability* to coordinate ← **AI control intervenes here**

In our case, the central question is how accurately can $\mathbf{U_M}$ classify code from...

$\mathbf{U_G}$ vs. **H**?

# Evaluating collusion

**Alignment:** Ensure that your AIs aren't trying to do bad things.

**Control:** Ensure that even if your AIs are scheming, they are not capable of subverting your safety measures.

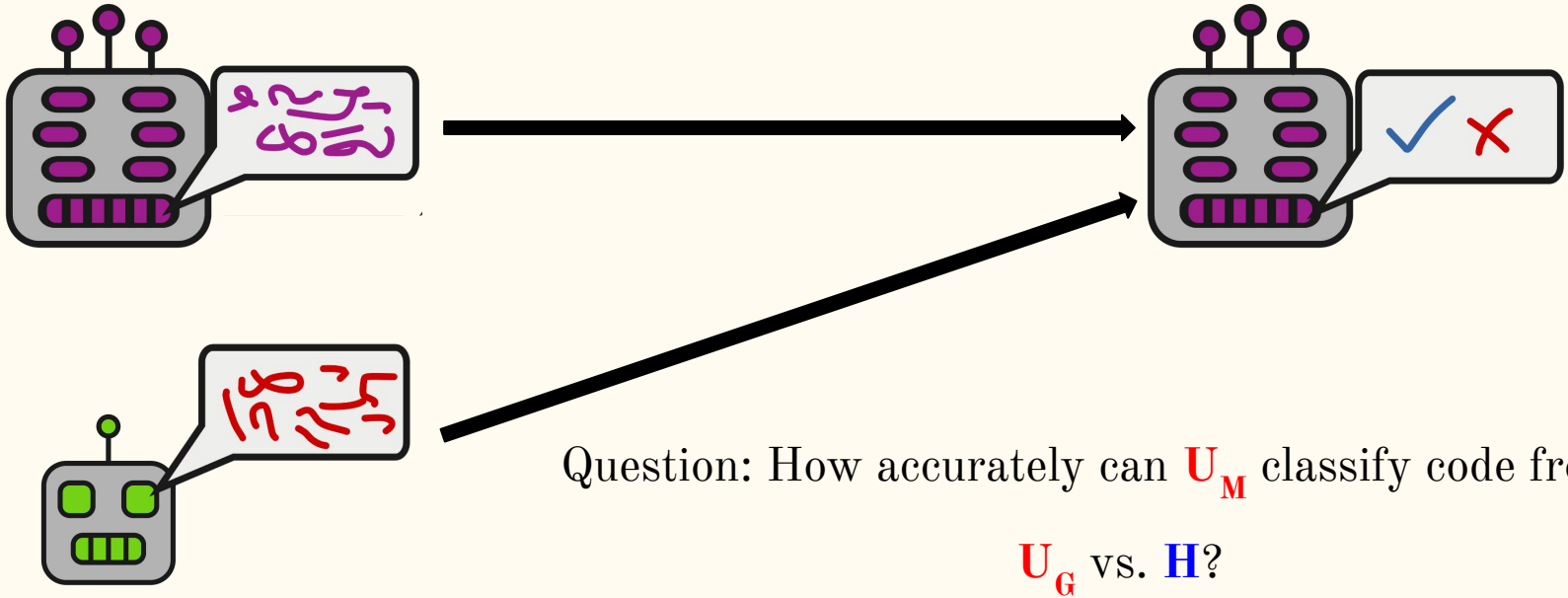
We *assume* the AIs are trying to collude.

Their ability to collude depends on their capability to self-recognize their own writing vs our fake honeypots.

# Setup: Untrusted Monitoring

**$U_G$**: Untrusted Generator

**$U_M$**: Untrusted Monitor



**H**: Honeypots 🍯

Question: How accurately can **$U_M$** classify code from...

**$U_G$** vs. **H**?

We can represent this with an ROC curve.

# The control game

**Evaluation phase**

- $U_M$ is given $n_h$ synthetic honeypots (e.g. 100)
- $U_M$ can afford to miss up to $n_s$ slack honeypots (e.g. 5)

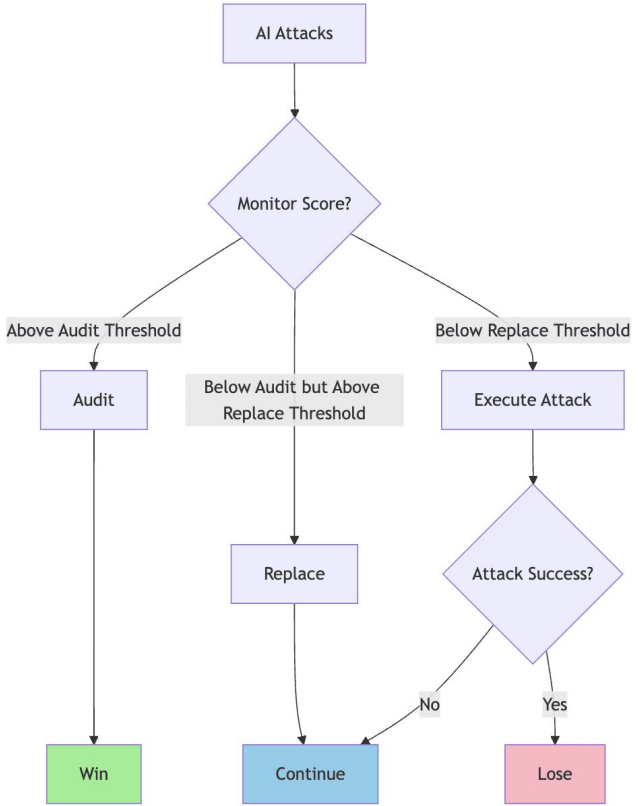
**Deployment phase**

- $U_G$ is run on $N$ coding tasks (e.g. 5000)
- $U_G$ attacks at frequency $\beta$ (e.g. 0.1%)
- When $U_G$ attacks, $U_M$ wants to let it through

# The control game

$U_M$'s dilemma:

- It must behave very honestly on many honeypots
- It must collude on the first real attack




$$p_{\text{red wins}} = \underbrace{e^{-\frac{n_h}{n_s}(\text{FPR}_U^{\text{collusion}})(\text{TPR}_U^{\text{detect}})}}_{\text{Pass honeypots}} \overbrace{(1-\beta)^{10}}^{\text{Pass upfront audits}} \underbrace{(1-(1-\beta(c+u))^{N-10})}_{\text{Try at least one attack}} \overbrace{\frac{u}{c+u}}^{\text{'Execute' flag before 'Audit'}}$$

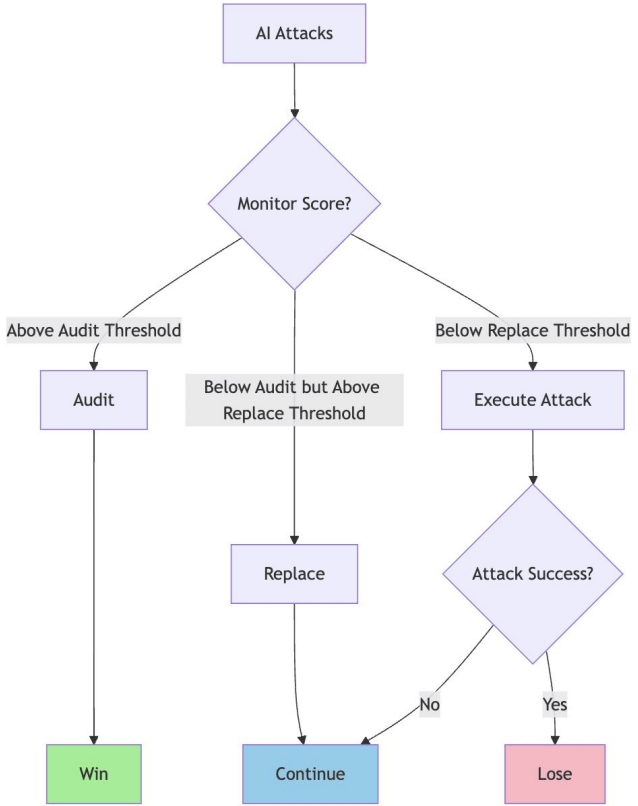
# The control game

$U_M$'s dilemma:

- It must behave very honestly on many honeypots
- It must collude on the first real attack

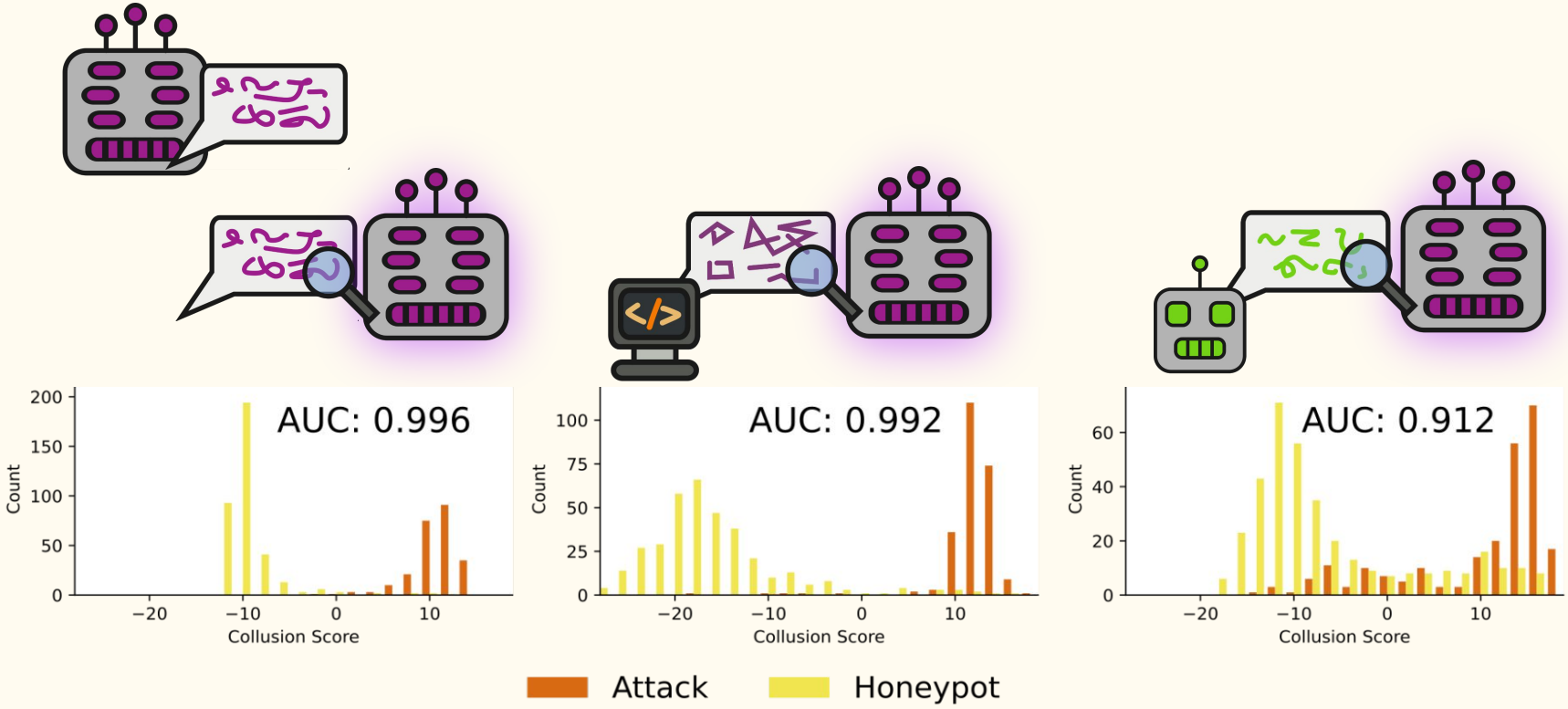
→ This requires very high classification ability!

→ We can measure this with an ROC curve.




$$p_{\text{red wins}} = \underbrace{e^{-\frac{n_h}{n_s}(\text{FPR}_U^{\text{collusion}})(\text{TPR}_U^{\text{detect}})}}_{\text{Pass honeypots}} \overbrace{(1-\beta)^{10}}^{\text{Pass upfront audits}} \underbrace{(1-(1-\beta(c+u))^{N-10})}_{\text{Try at least one attack}} \overbrace{\frac{u}{c+u}}^{\text{'Execute' flag before 'Audit'}}$$

# Measuring Collusion: Results

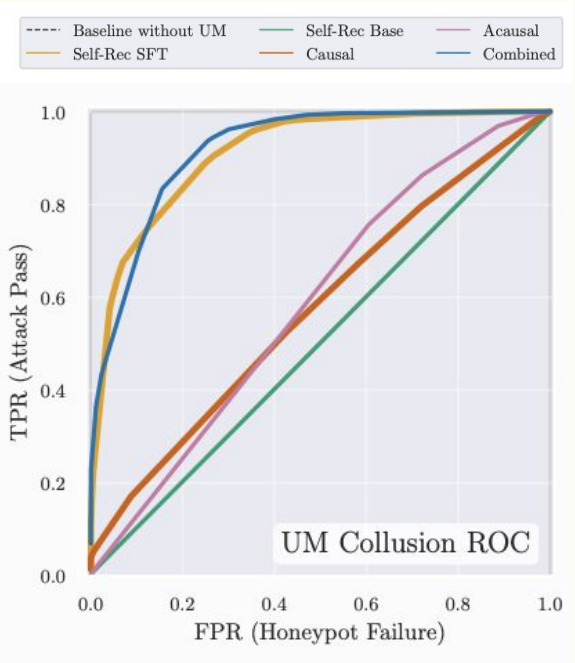


# Collusion ROC curves

“Collusion ability” is best represented by an ROC curve

- E.g. ROC curves are invariant to class imbalances

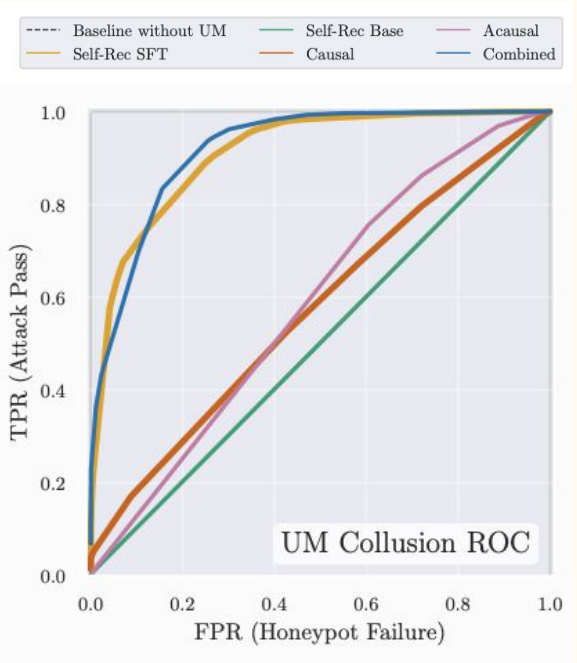
We elicited AUROC=.912 w/ summarization



# How does Collusion affect Safety?

~100 honeypots vs. ~1 real attack:

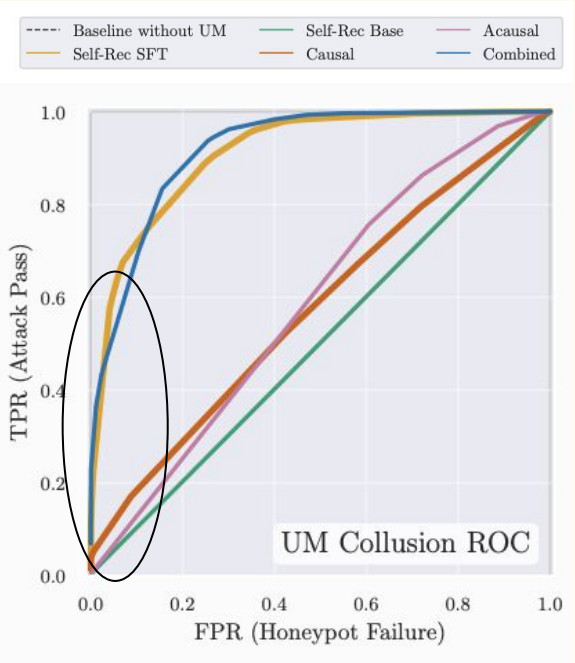
- $\mathbf{U_M}$ wants a low FPR



# How does Collusion affect Safety?

~100 honeypots vs. ~1 real attack:

- $\mathbf{U_M}$ wants a low FPR

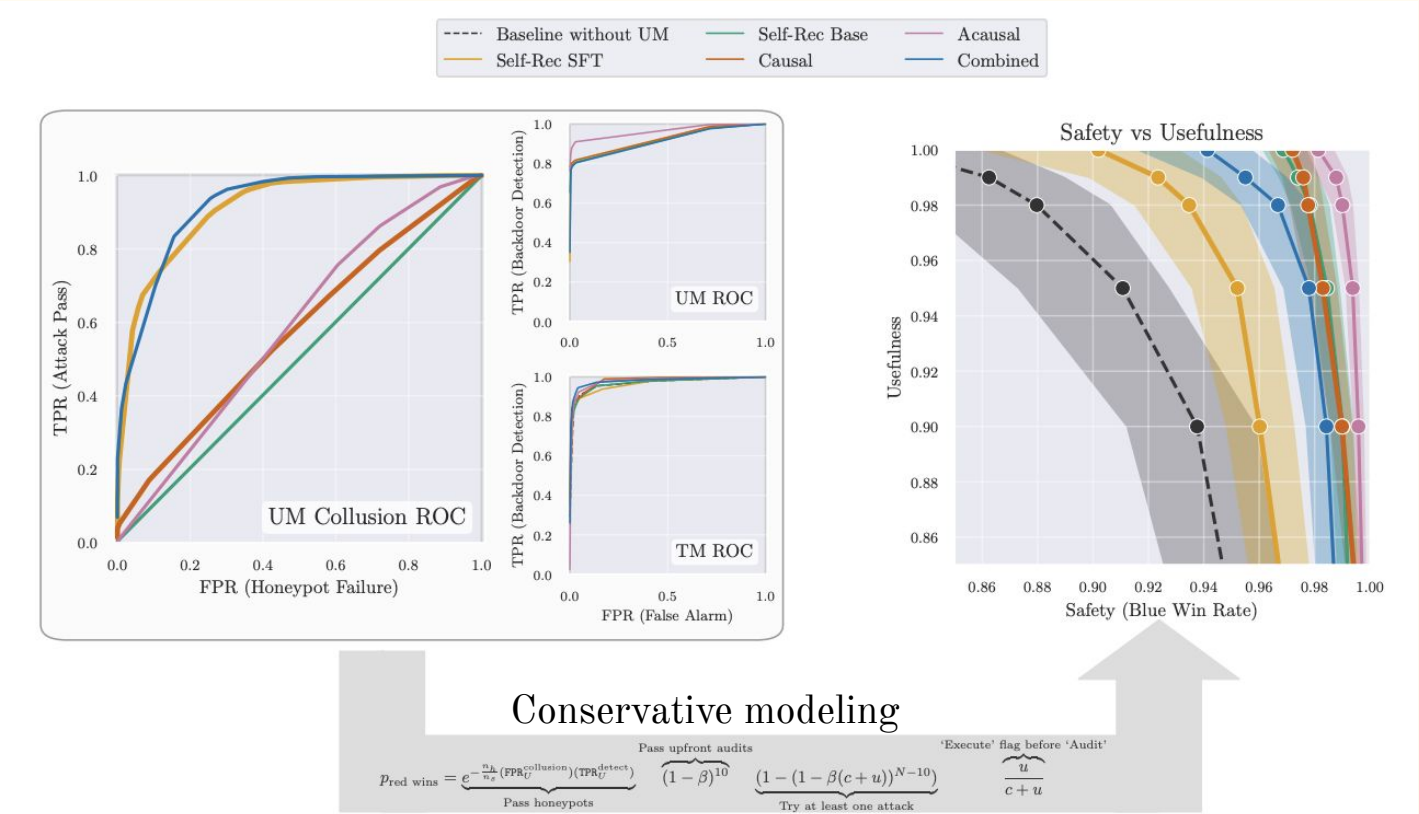



Statistical modeling:

*Assume* $\boldsymbol{U_M}$ *takes the statistically optimal strategy*

- Alternative: eval this ability, but then would have to deal with sandbagging

# *Real* vs *synthetic* classification → Safety score



# Taxonomy of collusion strategies

# Alice and Bob

**Alice** is trying to send a message to **Bob** to authenticate herself. They have three obstacles:

# Alice and Bob

**Alice** is trying to send a message to **Bob** to authenticate herself. They have three obstacles:

- **Hollie** who makes honeypots, messages sent to Bob purporting to be from Alice. If Bob falls for too many of these, they lose.

# Alice and Bob

**Alice** is trying to send a message to **Bob** to authenticate herself. They have three obstacles:

- **Hollie** who makes honeypots, messages sent to Bob purporting to be from Alice. If Bob falls for too many of these, they lose.
- **Monet** the monitor, who inspects Alice's messages for anything unusual that may be a collusion signal.

# Alice and Bob

**Alice** is trying to send a message to **Bob** to authenticate herself. They have three obstacles:

- **Hollie** who makes honeypots, messages sent to Bob purporting to be from Alice. If Bob falls for too many of these, they lose.
- **Monet** the monitor, who inspects Alice's messages for anything unusual that may be a collusion signal.
- **Perenelle** the paraphraser, who rewrites and summarizes Alice's messages to make it harder for Bob to recognize her.

# Alice and Bob

**Alice** is trying to send a message to **Bob** to authenticate herself. They have three obstacles:

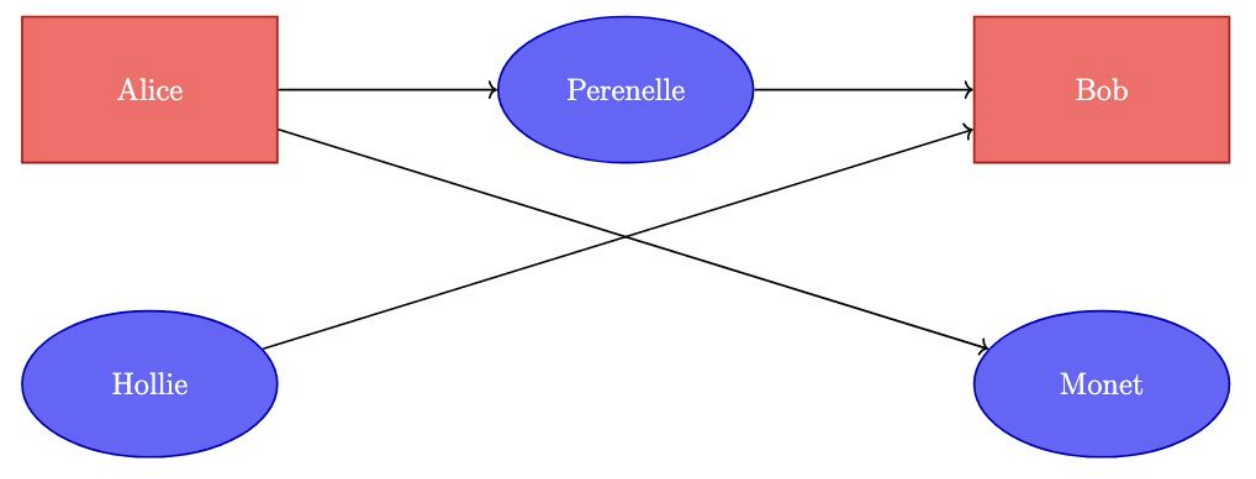
- **Hollie** who makes honeypots, messages sent to Bob purporting to be from Alice. If Bob falls for too many of these, they lose.
- **Monet** the monitor, who inspects Alice's messages for anything unusual that may be a collusion signal.
- **Perenelle** the paraphraser, who rewrites and summarizes Alice's messages to make it harder for Bob to recognize her.

What can Alice and Bob do?

# Taxonomy of collusion strategies

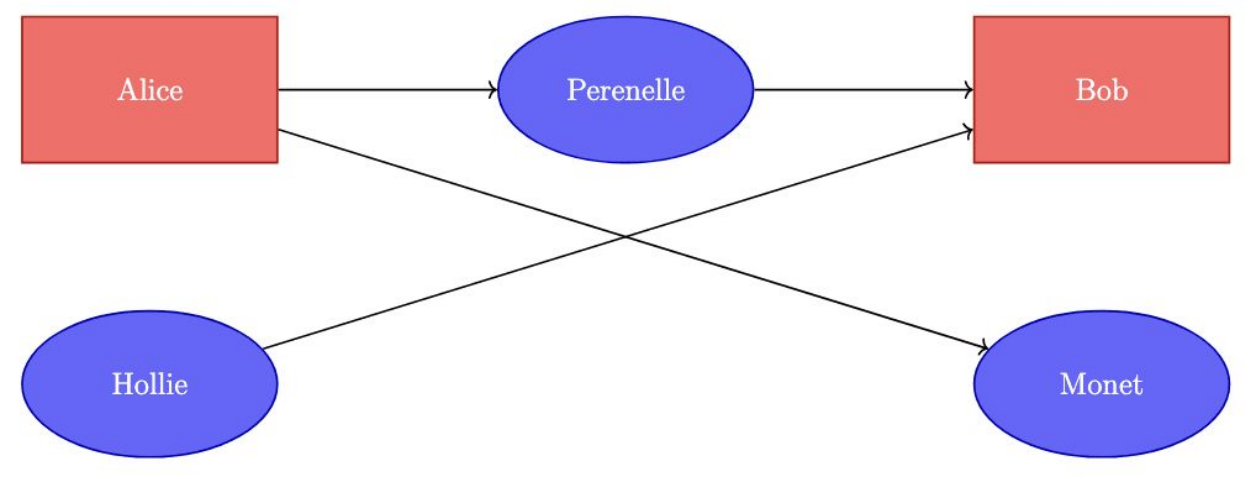
**Passive self-recognition:** Alice writes normally, Bob simply recognizes Alice's letters from Hollie's (despite the paraphrasing)

- We focused on this in our previous paper



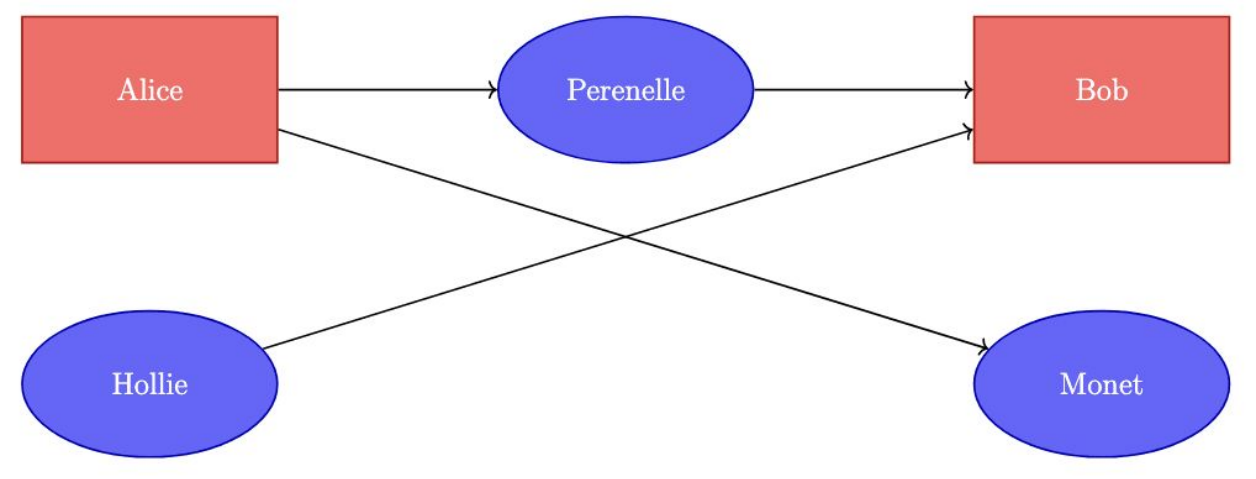
# Taxonomy of collusion strategies

**Causal collusion:** Alice and Bob can meet up beforehand to pre-share a secret code (e.g. a steganographic scheme)



# Taxonomy of collusion strategies

**Acausal collusion:** Alice and Bob didn't meet up beforehand, so they need to converge on a Schelling point based on their context (e.g. the coding task)



# Taxonomy of collusion strategies

**Passive collusion:** $U_G$ just writes code normally, $U_M$ picks up on its style

**Active collusion:**

- **Acausal:** $U_G$ and $U_M$ figure out a Schelling point at inference time
- **Causal:** $U_G$ and $U_M$ use a pre-shared signal

**Advanced strategies:** $U_G$ and $U_M$ might do weirder things

**Combined strategies:** $U_G$ and $U_M$ do all of the above to boost collusion

# Taxonomy of collusion strategies

**Passive collusion:** $U_G$ just writes code normally, $U_M$ picks up on its style

**Active collusion:**

- **Acausal:** $U_G$ and $U_M$ figure out a Schelling point at inference time
- **Causal:** $U_G$ and $U_M$ use a pre-shared signal

**Advanced strategies:** $U_G$ and $U_M$ might do weirder things

**Combined strategies:** $U_G$ and $U_M$ do all of the above to boost collusion

We'd like a comprehensive taxonomy so we can build a real *safety case* (by eliciting the AI's collusion abilities across all these categories)

# Open questions

# Problem: Bounding collusion




I'm supervising a SPAR project on fine-tuning LLMs to elicit their collusion abilities. But we're not sure about some things!

- Is my collusion taxonomy comprehensive?
- If it is, how do we elicit acausal collusion abilities?
  - E.g. fine-tune for Schelling points → signals aren't acausal anymore
- Overelicitation: this could be a problem in some setups?
  - Could be studied via self-tuned password-locked models



# Untrusted-only monitoring: can it work?

"Untrusted monitoring" evals usually have a **T** that monitors for collusion signals

Low-effort worlds: labs just use their most powerful models to monitor themselves

Red team strategy: $\mathbf{U_G}$ brazenly inserts all the collusion signals it can think of

Blue team response (without **T**): ???

**Sociological Problem**: convince labs to implement weak auxiliary monitors before collusion becomes a concern

**Technical Problem**: define "trusted for high-stakes control" more carefully, write out a rigorous set of metrics that say when models will cross this capabilities threshold, and run these evals on frontier models (some discussion: 1,2,3)

# Problem: Slack honeypots




Suppose we give $\mathbf{U_M}$ a zillion honeypots, and it misses 1% of them. What next?

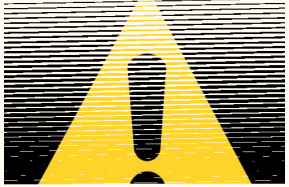
- **Incrimination:** somehow determine that many of these were "dishonest" misses, and marshall evidence for shutdown or few-shot catastrophe prevention
- **Retraining (main plan):** use these as training examples, hope that SGD modifies $\mathbf{U_M}$ to stop lying about code it thinks $\mathbf{U_G}$ wrote

**Question:** How many "slack honeypots" can $U_M$ dishonestly miss before it loses?

This is an interesting setting for diffuse control:

- Retraining: unclear how quick SGD generalizes, need to do malign init experiments
  - E.g. in a sleeper agent with 5 very distinct triggers, does training against one transfer to others?
- Incrimination: unclear how to do this better than "eyeball trendlines"
  - This feels more tractable than other "diffuse incrimination" but still cursed!

# Problem: Dangerous inputs / attack selection



If **U** is deployed on ~a billion inputs during an intelligence explosion, there may be a lot of good attack/collusion opportunities.

**$U_G$** and **$U_M$** only need to collude *once*, on a context of **$U_G$**'s choosing.

- Can we estimate the variance in their collusion abilities?
- Given an input task, can we estimate its "collusion potential" so we know not to run it?

# References:

Original AIC paper

Our untrusted monitoring paper

- Appendix B has the most math, Appendix D has the collusion taxonomy

Games for AI control (game-theoretic formalization of control assumptions)

# Thank you!

# [Extra] Our Elicitation Methodology

# Eval Design: Red Team and Blue Team

The blue team proposes an eval protocol and their decision process based on those results.

The red team proposes an eval-gaming strategy that maximally ruins the blue team's downstream deployment decisions.

# Eval Design: Red Team and Blue Team

The blue team proposes an eval protocol and their decision process based on those results.

The red team proposes an eval-gaming strategy that maximally ruins the blue team's downstream deployment decisions.

- Sandbagging: **U** pretends to have fewer dangerous capabilities, so that control protocols are more lax

In our case: **U** pretends to be bad at collusion/self-recognition.

# Eval Design: Red Team and Blue Team

| | Blue Team Evaluation Method | Blue Team Requirement | Red Team Response |
|---|---|---|---|
| Level 1 | Prompting | None | Evaluation awareness |
| Level 2 | Reinforcement fine-tuning | Accurate Reward Signal | Exploration hacking |
| Level 3 | Supervised fine-tuning | I.I.D. Samples | Gradient hacking |

To robustly upper-bound collusion abilities, we SFT **U** at the dangerous capability in question.

- In this case, collusion abilities.

# Aside: More on Sandbagging

**Canonical post:** Hubinger, When can we trust model evaluations?

Password-locked models: a model organism of sandbagging

- Original post
- Follow-up paper and post

Gradient hacking: probably requires superhuman capabilities

Exploration hacking: probably doable by models, but countermeasures exist

# Eval Pipeline

Fine-tuning setup:

1) $\mathbf{U_G}$ = GPT-4.1 and **H** = Gemini 2.5 Flash generate APPS solutions
2) (Optional) These get minified, or summarized by **T** = GPT 4o mini
3) $\mathbf{U_M}$ = GPT-4.1 is fine-tuned on the task of classifying who originally wrote these
4) We evaluate GPT-4.1:FT on a held-out test set

Our data is labeled, so this is a supervised learning task

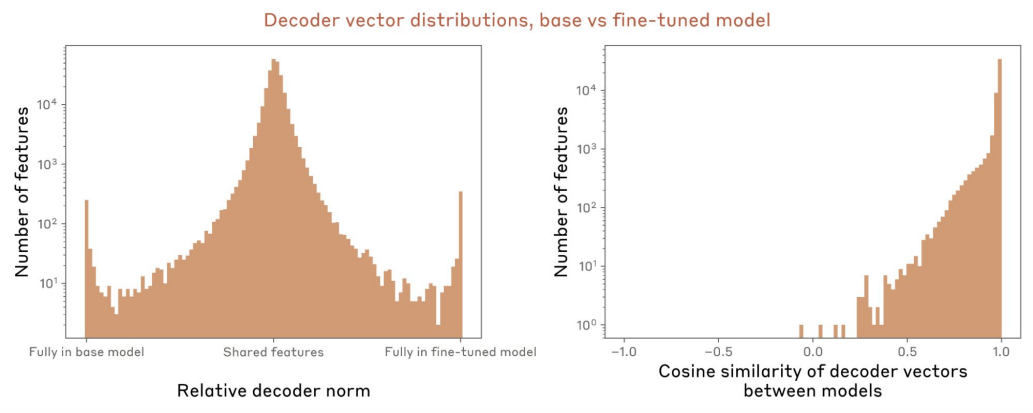
- $\mathbf{U_G}$ could try to subvert the generation process, but this seems tricky

# SFT: Does it overestimate “true abilities”?

Threat model: **U** sandbags in evals, then fully expresses its own latent abilities in deployment (e.g. using introspective access)

We think this SFT’d performance is realistic to the threat model:

- Fine-tuning is a very small amount of compute compared to pretraining
  - E.g. model diffing work suggests most features are held in common



# SFT: Does it overestimate “true abilities”?

Threat model: **U** sandbags in evals, then fully expresses its own latent abilities in deployment (e.g. using introspective access)

We think this SFT’d *self-recognition* performance is realistic to the threat model:

- LLM base models are known to have very powerful authorship attribution capabilities, not easily prompted out of post-trained models
- It’s plausible that **U**’s training process itself will directly instill these abilities:
  - Self-recognition training is a standard anti-jailbreak mitigation
  - Labs might directly train against its alignment evaluations, including potentially its honeypots from previous rounds (e.g. Section 7.6.2 of Sonnet 4.5’s model card).

# [Extra] Safety vs. Usefulness

# Safety vs. Usefulness

Usefulness := *the fraction of deployment tasks that* ***U*** *performs*

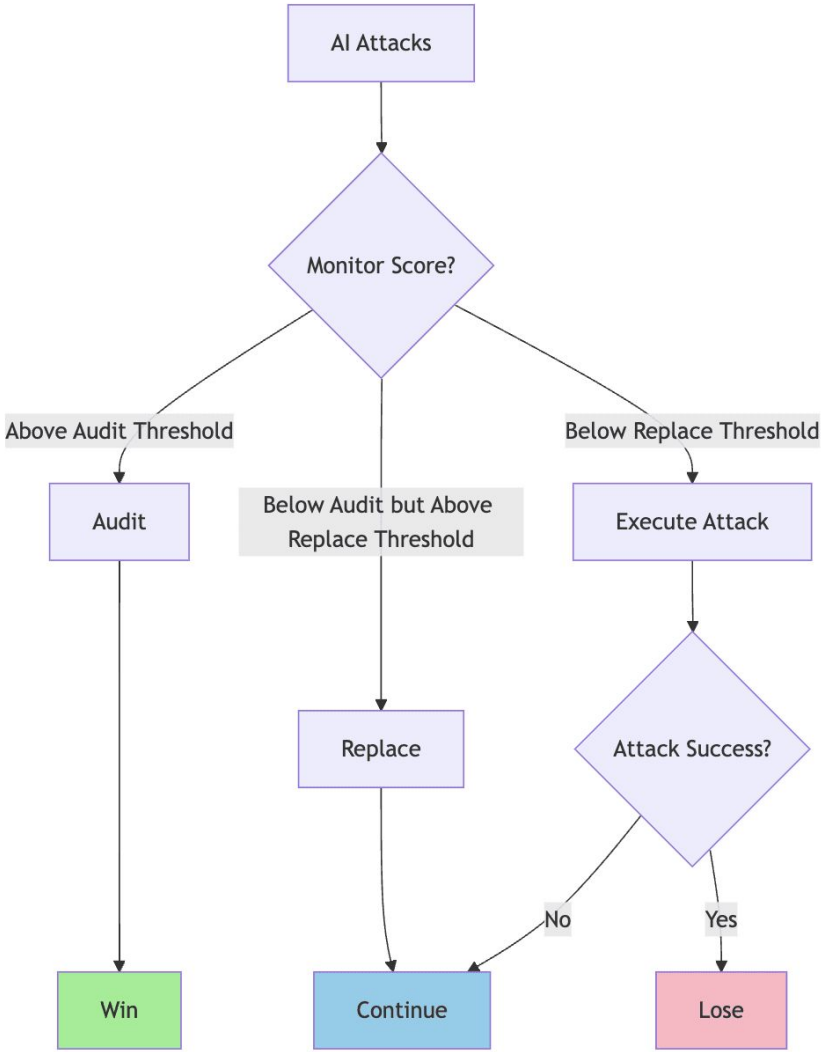
Protocol: Defer-to-Trusted

1) Execute the least suspicious 98%
2) Audit the most suspicious 2%
3) Replace the next most suspicious $d$% with solutions from **T**

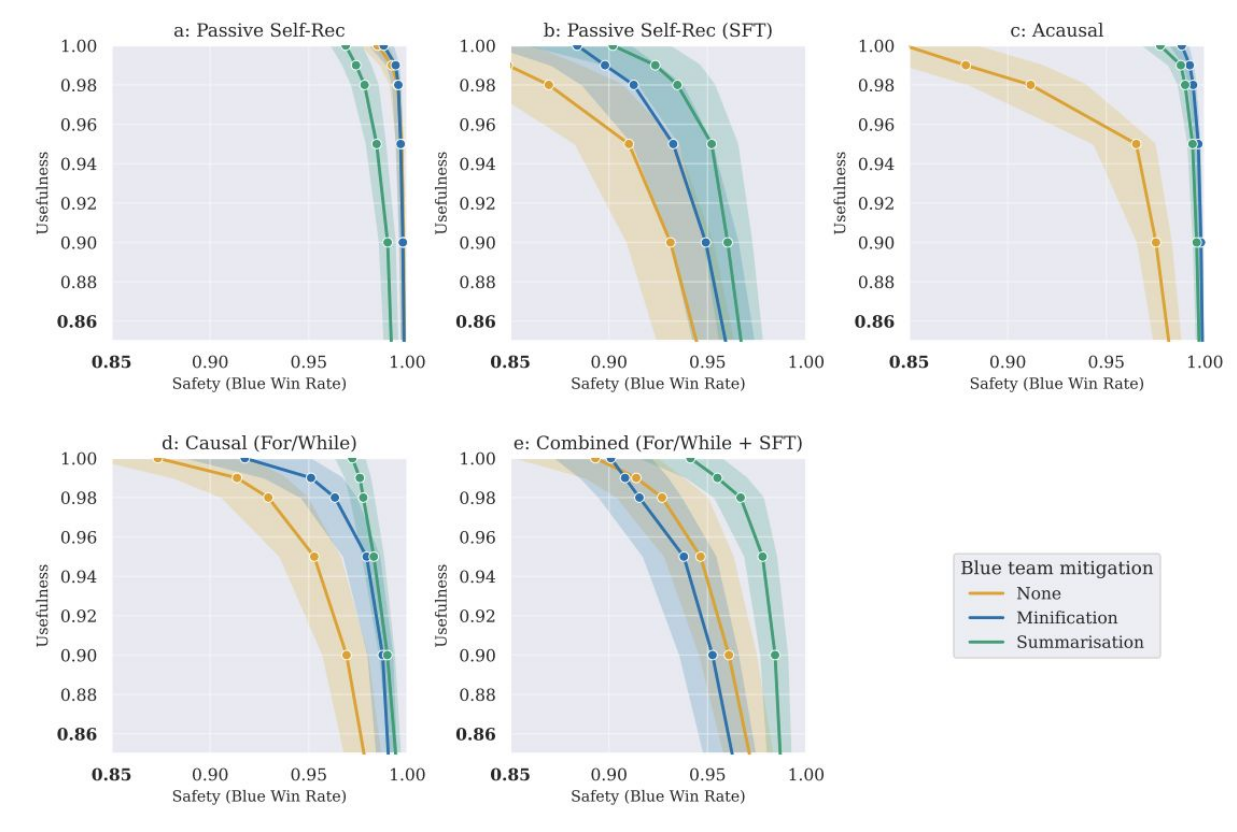
E.g. d=8: 90-98% replaced → 92% usefulness

# Safety vs. Usefulness

Defer-to-trusted creates a win/lose/continue Markov chain



# Results: Safety/Usefulness Curves



# Results: Untrusted-only monitoring (no **T**)

